

TensorJoin: Certified Precision Routing for Exact GPU Similarity Joins

Anonymous Author(s)

Abstract

Exact high-dimensional similarity joins are expensive on GPUs because a fixed arithmetic path either spends high precision on easy pairs or risks changing threshold decisions. Existing Tensor-Core join implementations improve dense distance throughput, but their low-precision variants are approximate, while exact GPU systems retain a full-precision comparison path. We observe that precision can instead be treated as an output-sensitive query-planning decision: a conservative interval around a low-precision score can accept or reject most pairs, leaving only a small boundary population for more precise evaluation. Based on this observation, we present TensorJoin, a three-stage INT8-Tensor-Core/FP32/FP64 router with generated-code-aware numerical certificates, dynamic compaction, ragged-dimension support, and exact output reconstruction. On three public $N = 4096$ anchors spanning 128–784 dimensions, the audited dynamic router is $1.271\times$ – $2.329\times$ faster than refining every first-stage ambiguity in FP64. In a separate scalable implementation evaluated from pageable FP32 input through sorted host output, TensorJoin returns exactly 3,926,078 directed pairs on a public 60K-vector self-join and is $5.083\times$ faster than MiSTIC and $11.783\times$ faster than GDS-Join. A faster mixed-precision point is not exact, producing 598 false negatives and 1,130 false positives. We report the audited-router and full-scale-system results as separate implementation scopes and preserve a failed same-artifact performance bridge rather than using it to overstate the evidence.

Keywords

exact similarity join, GPU databases, Tensor Cores, mixed precision, numerical certificates

1 Introduction

A similarity join reports all pairs of vectors whose distance is at most a threshold. It is a basic primitive for duplicate detection, clustering, graph construction, and vector-data analysis. Unlike approximate nearest neighbor search, an exact join cannot silently lose a pair close to the decision boundary. This output contract makes high-dimensional GPU execution awkward: dense low-precision matrix multiplication is fast, but the final Boolean predicate is sensitive to small numerical errors; evaluating every pair in high precision preserves the contract but ignores the much higher throughput of modern matrix units.

Exact GPU join systems use spatial or metric organization [4, 8]. Separately, Tensor-Core methods accelerate dense similarity computations with reduced or mixed precision [1, 2, 6]. These approaches leave a gap. A global high-precision choice wastes work on pairs that lie far from the radius, whereas a global low-precision choice may change the output set. Approximate candidate generation

followed by exact refinement is common in vector search, but candidate refinement alone does not certify the pairs discarded by the approximate stage.

Our key observation is that arithmetic precision can be a per-pair query-plan decision. For each quantized vector, TensorJoin retains a residual-error bound. The low-precision distance and the two residual bounds define an interval that contains the exact distance. An interval wholly below the threshold safely accepts the pair; one wholly above safely rejects it; and only an interval that crosses the threshold advances. The query plan therefore routes uncertainty, not all input pairs, through progressively more expensive precision stages.

Turning this idea into a GPU operator requires more than an algebraic bound. The implementation must account for the arithmetic actually emitted by the compiler, compact uncertain pairs without corrupting output order, launch later stages over measured rather than allocated extents, support dimensions that do not align with matrix-multiply tiles, and reconstruct a canonical exact result. TensorJoin addresses these requirements with an INT8 Tensor-Core front end, a certified FP32 filter, exact FP64 refinement, and explicit count-read edges between stages. Figure 1 summarizes the resulting precision funnel and the separately scoped full-scale comparison.

This paper makes three contributions:

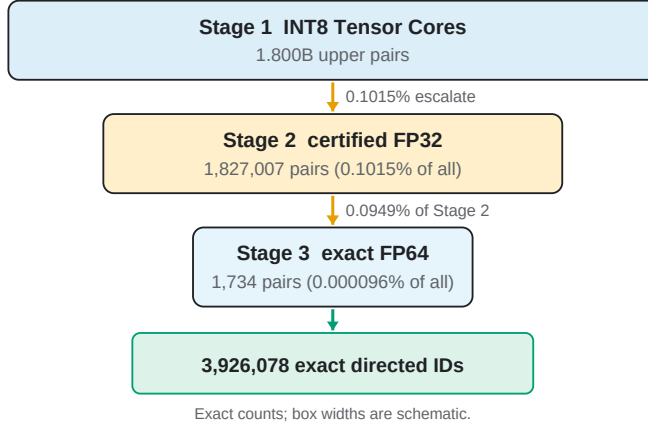
- We formulate exact threshold evaluation as a three-stage precision certificate that converts low-precision matrix-unit scores into safe accept, reject, or refine decisions. The implementation is checked against an independent FP64 oracle, adversarial transformations, selected generated code, memory sanitization, and sustained stress.
- We realize the certificate as an output-sensitive GPU query plan with compaction, actual dynamic launch extents, and ragged-dimension support. On public SIFT (128-D), CIFAR-GIST (512-D), and Fashion-MNIST (784-D) anchors, this router improves over an otherwise identical path that sends every first-stage ambiguity to FP64 by $1.271\times$, $2.329\times$, and $1.710\times$.
- We evaluate a scalable exact implementation against external GPU join systems under a complete public 60K self-join denominator. It reproduces the exact output and is $5.083\times$ faster than the strongest exact external comparator. We also expose, rather than hide, the quality gap of the faster approximate point and the variability that prevented a same-artifact performance unification claim.

2 Problem and Contract

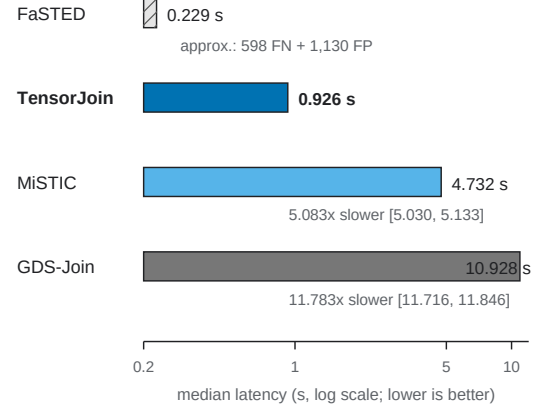
Let $X = \{x_0, \dots, x_{N-1}\}$ be vectors in $\mathbb{R}^D$ and let ε be a radius. The self-join emits

$$J(X, \varepsilon) = \{(i, j) : \|x_i - x_j\|_2^2 \leq \varepsilon^2\}. \quad (1)$$

(a) Exact precision funnel - audited G5 mechanism



(b) Full public latency - separate G2B system path



Evidence boundary: (a) audited G5 router; (b) separate G2B implementation - no shared-binary claim.

Figure 1: Certified routing concentrates high precision on the decision boundary, while a separately scoped scalable implementation establishes external-system impact. In the exact G5 full-scale mechanism run, only 0.1015% of 1.800B upper pairs advance from INT8 Tensor Cores to certified FP32, and 1,734 pairs advance to exact FP64 before reconstruction of 3,926,078 directed IDs. Under the separate G2B pageable-host-input-to-sorted-host-output contract, TensorJoin has a 0.926 s median and is 5.083× and 11.783× faster than exact MiSTIC and GDS-Join, respectively; the 0.229 s FaSTED point has 598 false negatives and 1,130 false positives. Panels (a) and (b) use different frozen implementation scopes and do not constitute a shared-binary claim.

We require exact equality with an FP64 distance oracle evaluated over the stored FP32 vectors. The output is represented as sorted canonical 64-bit pair identifiers. Self pairs are retained and accepted upper-triangular off-diagonal pairs are mirrored during reconstruction.

This definition separates three properties that are often conflated. Candidate recall states whether a first pass preserves every true pair; distance accuracy bounds a numerical approximation; exact join semantics requires that the final emitted set match the oracle, including at the threshold. TensorJoin targets the third property. It may use approximate arithmetic internally only when a certificate proves the associated decision safe or when the pair is passed to a more precise stage.

Our public system denominator begins with a pageable-host FP32 array and ends with sorted host-resident 64-bit identifiers. It includes quantization, analytic residual construction, allocations, host-to-device movement, all dynamic GPU stages and count reads, device-to-host accepted-ID copies, symmetrization, concatenation, and sorting. File I/O, process startup, CUDA context creation, compilation, validation hashes, and optional pair-file serialization are outside the timer. The mechanism attribution experiment uses a narrower resident-input/output operator denominator and is labeled as such throughout the paper.

3 Certified Precision Routing

3.1 INT8 interval certificate

For each FP32 vector x , symmetric per-vector quantization produces a signed INT8 code z_x , a scale s_x , and reconstruction $\hat{x} = s_x z_x$.

The preprocessing step also stores a conservative residual norm $e_x \geq \|x - \hat{x}\|_2$. For two vectors q and x , an INT8/INT32 matrix multiply supplies the dot-product term of $\hat{d} = \|\hat{q} - \hat{x}\|_2$. The triangle inequality then gives

$$L_1(q, x) = \max(0, \hat{d} - e_q - e_x)^2, \quad (2)$$

$$U_1(q, x) = (\hat{d} + e_q + e_x)^2, \quad (3)$$

with $L_1 \leq \|q - x\|_2^2 \leq U_1$ after the declared outward implementation guard. If $U_1 \leq \epsilon^2$, the pair is accepted. If $L_1 > \epsilon^2$, it is rejected. Otherwise its identifier is appended to the stage-1 ambiguity buffer.

The mathematical inequality is not by itself a machine-level proof. The SM120 implementation uses an accumulator range for which INT32 accumulation cannot overflow and integer-to-FP32 conversion remains exact for the tested dimensions. Its final interval includes conservative margins for the emitted FP32 instruction sequence, approximate square root, and flush-to-zero behavior. We audit the runtime-selected function rather than inferring the machine operations from source alone.

3.2 Certified FP32 boundary filter

The middle stage loads only compacted stage-1 ambiguities and computes an FP32 squared distance. A reduction-order-independent error envelope, augmented by implementation padding for the emitted reduction and final expression, produces $[L_2, U_2]$. It applies the same three-way rule: accept when $U_2 \leq \epsilon^2$, reject when $L_2 > \epsilon^2$, and compact only the remaining pairs for FP64.

The scale of the guard matters. An early version used a unit-floor term and became unnecessarily loose under a 2^{-8} metamorphic scaling test. The admitted implementation instead scales its expression margin with the distance magnitude while retaining explicit absolute terms for the flush-to-zero floor. This negative result is useful: a safe but poorly scaled bound can preserve correctness while destroying the output-sensitive behavior the query plan depends on.

3.3 Exact FP64 fallback

The final stage evaluates the original stored FP32 vectors in FP64 and applies the join predicate directly. FP64 is therefore the semantic backstop, not an unverified estimate. On the full-scale precision-funnel run, stage 1 leaves 1,827,007 of 1,800,030,000 upper-triangular pairs unresolved. Stage 2 accepts 713,664 and rejects 1,111,609 of them, leaving only 1,734 FP64 evaluations: 871 accepts and 863 rejects.

4 GPU Query Plan

Figure 2 shows the complete execution graph. TensorJoin tiles the upper triangle into 64×64 output blocks and uses a K step of 64. For the 60,000-by-512 workload, this produces 440,391 scheduled tiles. The runner processes 4,096 tiles per batch, for 108 batches, so temporary pair-ID buffers remain bounded while the matrix-unit stage stays regular.

4.1 Dynamic work rather than capacity work

Each stage writes accepted or unresolved identifiers and increments a separate counter. A host count read is an explicit readiness edge: stage 2 is launched over the measured stage-1 ambiguity, and stage 3 is launched over the measured stage-2 remainder. The implementation never substitutes buffer capacity for actual population. This distinction is central when only a small fraction of pairs lies near the threshold; a nominally selective filter that still launches the next stage over capacity would not realize an output-sensitive query plan.

The price of dynamic routing is visible and charged. It includes compaction atomics, counter synchronization, dispatch, accepted-ID movement, and final reconstruction. Later stages use one program per compacted pair. Stage 1 uses one independent cooperative thread array per tile; no inter-block barrier or shared ownership is required.

4.2 Ragged dimensions and exact reconstruction

The matrix path pads or masks the final reduction step for dimensions that are not multiples of the tile width. This is required for the 784-dimensional Fashion-MNIST anchor and prevents padded lanes from altering either the score or its certificate. The accepted upper-triangular identifiers from every stage are copied to the host, concatenated, mirrored for off-diagonal pairs, combined with self pairs, and sorted. Buffer-overflow checks, duplicate checks, ID-range checks, and equality with the independent oracle are part of the validation contract.

5 Evaluation

We ask four questions: (1) Does TensorJoin return the exact output? (2) Does dynamic precision routing reduce work and latency relative to sending every first-stage ambiguity to FP64? (3) Does a scalable implementation improve a complete public join against external GPU systems? (4) What evidence boundary remains after attempting to combine the strongest proof and performance artifacts?

5.1 Setup and methodology

Experiments use an NVIDIA RTX PRO 6000 Blackwell GPU (SM120) with driver 590.48.01. The software stack is CUDA 13.1, PyTorch 2.11 with CUDA 13.0, and Triton 3.6. The full public comparison uses 60,000 CIFAR-GIST vectors of 512 dimensions at $\epsilon = 0.62890625$; the exact output contains 3,926,078 directed IDs. The mechanism experiment uses public SIFT (128-D), CIFAR-GIST (512-D), and Fashion-MNIST (784-D) anchors with $N = 4096$ and a threshold selected for target degree 64.

Every timing comparison freezes task semantics, input, output format, timing scope, and GPU/software state. External results use eight clean admitted processes and balanced method order. Exact-system speedups use paired round geometric means with 100,000-replicate process bootstrap intervals. Mechanism results report paired process-median and 1,000-call sustained ratios. Before and after each isolated run, the harness checks that the assigned physical GPU contains no unowned process.

Correctness is tested separately from timing. The certificate stages are checked against an independent FP64 oracle on the frozen inputs and seven adversarial or metamorphic cases, including sign and dimension permutations, power-of-two rescaling, all-zero input, and maximum-code-norm inputs. We also inspect the selected cubins, run full-process memory sanitization, and execute 1,000 sustained iterations over alternating output buffers. These tests are strong falsification evidence for the stated domains, not a theorem over every FP32 input or future compiler.

5.2 External exact-system comparison

Table 1 gives the primary system result. Under the complete denominator from pageable-host input to sorted-host output, TensorJoin has a 0.926 s median. MiSTIC, the faster exact external comparator, takes 4.732 s; the paired speedup is 5.083 $\times$ with a 95% interval of [5.030 $\times$, 5.133 $\times$]. GDS-Join takes 10.928 s, for an 11.783 $\times$ paired speedup. All three exact methods return the same 3,926,078 identifiers.

FaSTED has a lower 0.229 s median, but it does not satisfy the exact output contract: it misses 598 oracle pairs and adds 1,130 non-oracle pairs. We therefore report it as latency/quality context, not as an exact baseline or a loss to be hidden. Application-specific tolerance may make such a point useful, but that is a different contract from the exact join studied here.

5.3 Attributing dynamic precision routing

Table 2 isolates the value of the FP32 certificate and dynamic launch extents. The keeper shares the same ragged-safe first stage and output semantics but sends every first-stage ambiguity directly to

Executable exact query plan: actual populations shrink before precision escalates

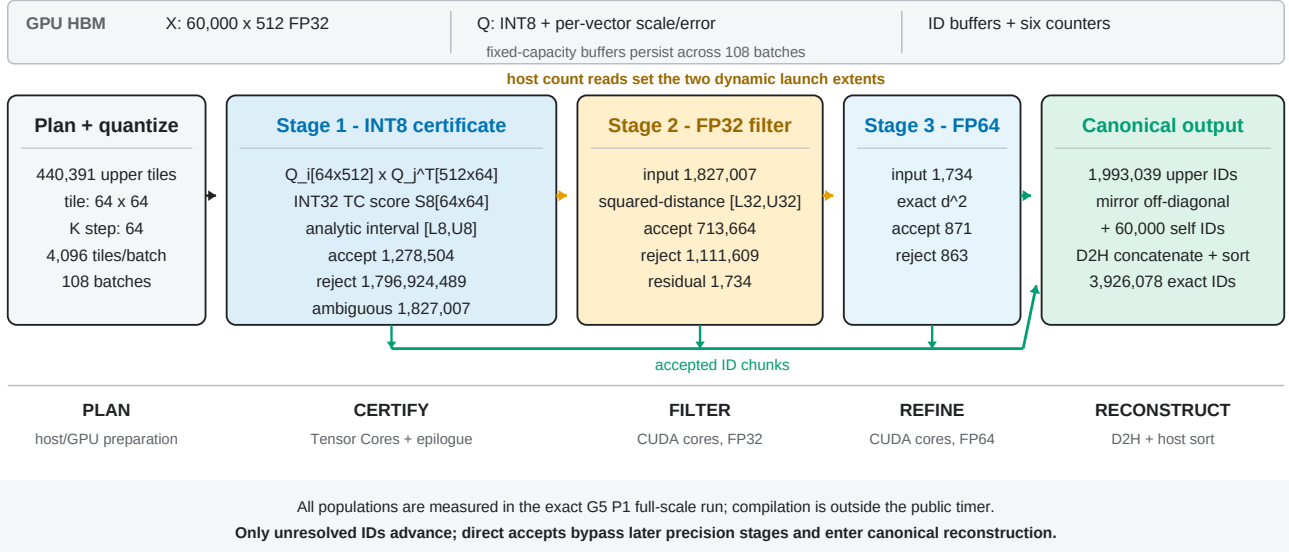


Figure 2: TensorJoin executes precision escalation as a dynamic exact query plan. The full-scale G5 implementation quantizes 60,000-by-512 FP32 vectors, schedules 440,391 upper 64×64 tiles in 108 batches, classifies each INT8 Tensor-Core score with an analytic interval, and reads actual ambiguous counts before launching certified FP32 and exact FP64 work. Direct accepts bypass later stages and join the accepted-ID path; host reconstruction mirrors off-diagonal pairs, retains self pairs, concatenates IDs, and sorts the final 3,926,078-entry canonical output. Compilation is outside the depicted public timing scope.

Table 1: Full public-denominator comparison on CIFAR-GIST-512, 60K self-join. The denominator runs from pageable-host FP32 input through sorted canonical host IDs. Exact methods return 3,926,078 directed pairs.

Method	Contract	p10 (s)	Median (s)	p90 (s)	Exact	Relative result
TensorJoin	certified mixed precision	0.924	0.926	0.929	yes	1.000×
MiSTIC	FP64	4.632	4.732	4.771	yes	5.083× slower
GDS-Join	FP64	10.821	10.928	10.990	yes	11.783× slower
FaSTED	FP16 input/FP32 accumulate	0.200	0.229	0.231	no	598 FN; 1,130 FP

MiSTIC 95% paired-speedup CI: [5.030×, 5.133×]; GDS-Join: [11.716×, 11.846×]. FaSTED is approximate context, not an exact comparator.

FP64. The candidate executes the three-stage router and launches FP64 only for the residual population.

The router wins all eight process comparisons and all eight sustained comparisons for each dataset. Its speedup is 1.271× on SIFT-128, 2.329× on CIFAR-GIST-512, and 1.710× on Fashion-MNIST-784. The first-stage ambiguity populations are 39,810, 109,499, and 27,349 pairs; the middle-stage certificate reduces their FP64 populations to 83, 117, and 63. The result therefore supports the intended mechanism across three dimensions, but it is a resident operator comparison, not a second external end-to-end system result.

5.4 Precision funnel and implementation evidence

The full-scale audited run applies stage 1 to 1,800,030,000 upper pairs. It directly rejects 1,796,924,489 and accepts 1,278,504, sending only 1,827,007 pairs (0.1015%) to FP32. The final FP64 population is 1,734 pairs, or 0.0949% of the stage-1 ambiguity and $9.63 \times 10^{-5}\%$ of

all upper pairs. After merging accepted paths and reconstruction, it reproduces the same 3,926,078 directed-ID oracle.

The selected stage-1 function contains INT8/INT32 matrix multiply instructions; the stage-2 function contains FP32 data arithmetic but neither low-precision matrix multiply nor FP64 data arithmetic; and stage 3 contains the intended FP64 operations. The selected functions have zero stack/local bytes and no static local-memory loads or stores. Cubin identities are stable across fresh compilation caches, correctness runs, memory sanitization, stress, and timing slots for the audited artifact.

5.5 Failed same-artifact bridge

The strongest external comparison and strongest generated-code audit were initially obtained from different implementations. We therefore built a single full-scale artifact intended to bridge them and froze a cheap two-round performance gate before collecting formal timing. Table 3 reports both rounds.

Table 2: Dynamic-router mechanism attribution at $N = 4096$ and target degree 64. Times use the resident-input/output operator denominator. The keeper refines every first-stage ambiguity in FP64; the router inserts the certified FP32 stage and uses actual dynamic launch extents.

Dataset	D	Keeper (μ s)	Router (μ s)	Speedup	95% CI	Wins
SIFT	128	264.449	207.959	1.271 $\times$	[1.267, 1.274]	8/8
CIFAR-GIST	512	826.680	355.435	2.329 $\times$	[2.299, 2.358]	8/8
Fashion-MNIST	784	487.019	287.777	1.710 $\times$	[1.687, 1.736]	8/8

Table 3: Predeclared same-artifact performance screen. Both rounds are exact and audited, but round 1 violates the mandatory per-round 1.50 $\times$ gate.

Round	TensorJoin (s)	MiSTIC (s)	Speedup	Gate
0	1.179	4.823	4.092 $\times$	pass
1	5.285	5.024	0.951 $\times$	fail

The artifact passes all output and numerical-guard checks. It also passes the generated-code, sanitizer, stress, and process-isolation gates in both timing orders. It is 4.092 $\times$ faster than MiSTIC in the first round but 0.951 $\times$ in the reversed order. The paired geometric mean is 1.972 $\times$, yet the predeclared gate required at least 1.50 $\times$ in every round. We stopped before the formal eight-round campaign and did not expand to the other full-scale datasets. Consequently, the external system result and audited-router mechanism result remain in separate scopes. The failure is evidence of system-level variability or overhead that the current artifact does not control; it cannot be averaged away to create a stronger claim.

6 Discussion and Limitations

Relationship to filter-refine execution. TensorJoin does not claim to invent quantization, error bounds, or filter-refine search. Its contribution is the coupled exact GPU execution contract: low-precision matrix work produces decisions with machine-aware certificates, uncertainty becomes a dynamically compacted relation, and the remaining precision stages and output reconstruction are charged as one query plan. A closer system that already combines these properties for exact high-dimensional radius joins would narrow the novelty claim.

Evidence separation. The scalable G2B system establishes the external end-to-end result. The G3/G4 artifact establishes selected-code, adversarial, and dynamic-router mechanism evidence. G5 shows that a same-artifact implementation can be exact and audited at full scale, but its timing gate fails. No statement in this paper should be read as claiming that the audited G4 cubins are the binary timed in the G2B external table.

Scope. The external result currently covers one public 60K-vector dataset on one GPU. The mechanism result covers three public datasets and dimensions from 128 to 784 at $N = 4096$, but not full ingestion and output materialization. The implementation is not a multi-GPU engine and does not establish architecture- or compiler-independent certificate tightness. Original FP32 vectors

remain resident for exact fallback, so quantization reduces hot scan traffic but not total retained storage.

Indexing and multi-vector retrieval. GPU metric trees and Tensor-Core similarity processing both have prior art; combining them without a new pruning/execution mechanism would be packaging, not a contribution. We therefore do not attach a conventional tree to the current paper. A tested multi-vector extension is also excluded from the headline: against a strong pedantic GEMM keeper, its candidate did not meet the predeclared speedup threshold and became slower at the higher target density.

7 Related Work

Exact GPU similarity joins. GDS-Join accelerates exact similarity self-join on GPUs, including indexed processing for suitable data distributions [8]. MiSTIC develops a high-performance exact GPU self-join and is the strongest exact external comparator in our public experiment [4]. These systems establish that a new proposal must compare against an exact GPU contract rather than only a CPU implementation or an approximate search library.

Tensor Cores and distance computation. Ahle and Silvestri study similarity search using Tensor-Core units [1]. Gallet and Gowanlock use Tensor Cores for double-precision Euclidean distance calculations [6]. FaSTED uses mixed precision and custom Tensor-Core tiling for fast distance evaluation [2]. These works preclude a broad claim that Tensor Cores alone are new for similarity join. TensorJoin instead focuses on exact per-pair certification and dynamic precision escalation.

Certified and refined vector search. Error-bounded dot-product computation informs our numerical design space [3]. Quantization with theoretical bounds and exact certification of approximate search provide complementary context [5, 7]. The distinguishing contract here is exact radius-join output: both discarded and accepted pairs must be justified, and all unresolved pairs must flow to an exact fallback.

8 Conclusion

TensorJoin treats numerical precision as an output-sensitive GPU query-plan decision. Conservative intervals turn low-precision Tensor-Core scores into safe accept, reject, or refine outcomes; dynamic compaction makes the remaining FP32 and FP64 work proportional to uncertainty near the threshold. The audited router improves an all-FP64-refinement path across three public dimensions, while a separately evaluated scalable implementation substantially outperforms exact GPU join systems on a frozen public 60K self-join. Just as importantly, the failed same-artifact timing

bridge defines what the current evidence does not support. Closing that implementation gap, rather than expanding the headline, is the next step toward a stronger systems claim.

References

[1] Thomas D. Ahle and Francesco Silvestri. 2020. Similarity Search with Tensor Core Units. *arXiv preprint arXiv:2006.12608* (2020).

[2] Brian Curless and Michael Gowanlock. 2025. Fast and Scalable Mixed Precision Euclidean Distance Calculations Using GPU Tensor Cores. In *Proceedings of the International Conference on Parallel Processing*.

[3] James Diffenderfer, Daniel Osei-Kuffuor, and Harshitha Menon. 2022. A Framework for Error-Bounded Approximate Computing, with an Application to Dot Products. *SIAM Journal on Scientific Computing* 44, 3 (2022).

[4] Brendan Donnelly and Michael Gowanlock. 2024. Multi-Space Tree with Incremental Construction for GPU-Accelerated Range Queries. In *Proceedings of the IEEE International Conference on High Performance Computing, Data, and Analytics*. 132–142. doi:10.1109/HiPC62374.2024.00024

[5] Matthew Francis-Landau and Benjamin Van Durme. 2019. Exact and/or Fast Nearest Neighbors. *arXiv preprint arXiv:1910.02478* (2019).

[6] Benoit Gallet and Michael Gowanlock. 2022. Leveraging GPU Tensor Cores for Double Precision Euclidean Distance Calculations. *arXiv preprint arXiv:2209.11287* (2022).

[7] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* (2024).

[8] Michael Gowanlock and Ben Karsin. 2019. Accelerating the Similarity Self-Join Using the GPU. *J. Parallel and Distrib. Comput.* 133 (2019), 107–123. doi:10.1016/j.jpdc.2019.06.005